

CS 기초

2026 Summer YBIGTA

28기 김지호

목차

- CS란?
- 컴퓨터 시스템
- 실습

CS란?

CS란?

- 컴퓨터 과학은 계산, 정보, 자동화와 이를 수행하는 컴퓨터 및 계산 시스템을 연구하는 학문이다. 컴퓨터 과학은 계산이 무엇인지, 어떤 문제가 계산 가능한지, 계산을 얼마나 효율적으로 수행할 수 있는지, 그리고 그러한 계산을 실제 시스템으로 어떻게 구현할 수 있는지를 다룬다.
 - https://ko.wikipedia.org/wiki/컴퓨터_과학

CS란?

- 계산(Computation) : 문제를 단계로 쪼개어 연산으로 해결한다.
- 정보(Information) : 정보를 컴퓨터가 이해하고 처리할 수 있는 형태로 표현한다.
- 자동화(Automation) : 이러한 과정을 사람의 개입 없이 수행하도록 만든다.

CS란?

- 계산(Computation) : 문제를 단계로 쪼개어 연산으로 해결한다.
 - 어떤 문제가 원리적으로 풀 수 있는 문제인가? (계산 가능성 이론)
 - 그렇다면 효과적인 풀이에는 무엇이 있을까? (알고리즘, 자료구조)
- 정보(Information) : 정보를 컴퓨터가 이해하고 처리할 수 있는 형태로 표현한다.
 - 컴퓨터가 정보를 어떻게 이해하게 할 수 있을까? (정보 이론)
- 자동화(Automation) : 이러한 과정을 사람의 개입 없이 수행하도록 만든다.
 - 실제 시스템으로 어떻게 구현할 수 있을까? (하드웨어 이론)

CS란?

- 계산(Computation) : 문제를 단계로 쪼개어 연산으로 해결한다.
 - 어떤 문제가 원리적으로 풀 수 있는 문제인가? (계산 가능성 이론)
 - 그렇다면 효과적인 풀이에는 무엇이 있을까? (알고리즘, 자료구조)
- 정보(Information) : 정보를 컴퓨터가 이해하고 처리할 수 있는 형태로 표현한다.
 - 컴퓨터가 정보를 어떻게 이해하게 할 수 있을까? (정보 이론)
- 자동화(Automation) : 이러한 과정을 사람의 개입 없이 수행하도록 만든다.
 - 실제 시스템으로 어떻게 구현할 수 있을까? (컴퓨터 시스템)

CS란?

- 실제 시스템으로 어떻게 구현할 수 있을까? (컴퓨터 시스템)
 - 컴퓨터 아키텍처
 - 운영체제
 - Windows
 - Linux
 - Mac OS
 - 네트워크
 - 데이터베이스
 - 컴파일러
 - ...

CS란?

- 컴퓨터 시스템
 - 운영체제
 - Linux
 - 가상화
 - Shell Scirpt

CS란?

- 이걸 왜 배워야 하나요?
- Claude Code / Codex가 다 해주는거 아님?
- 오히려 코드를 짜는 행위 자체는 LLM이 해주니까, 전반적인 구조에 대한 이해가 있어야 더욱 효율적인 개발이 가능하다

컴퓨터 시스템

컴퓨터 시스템

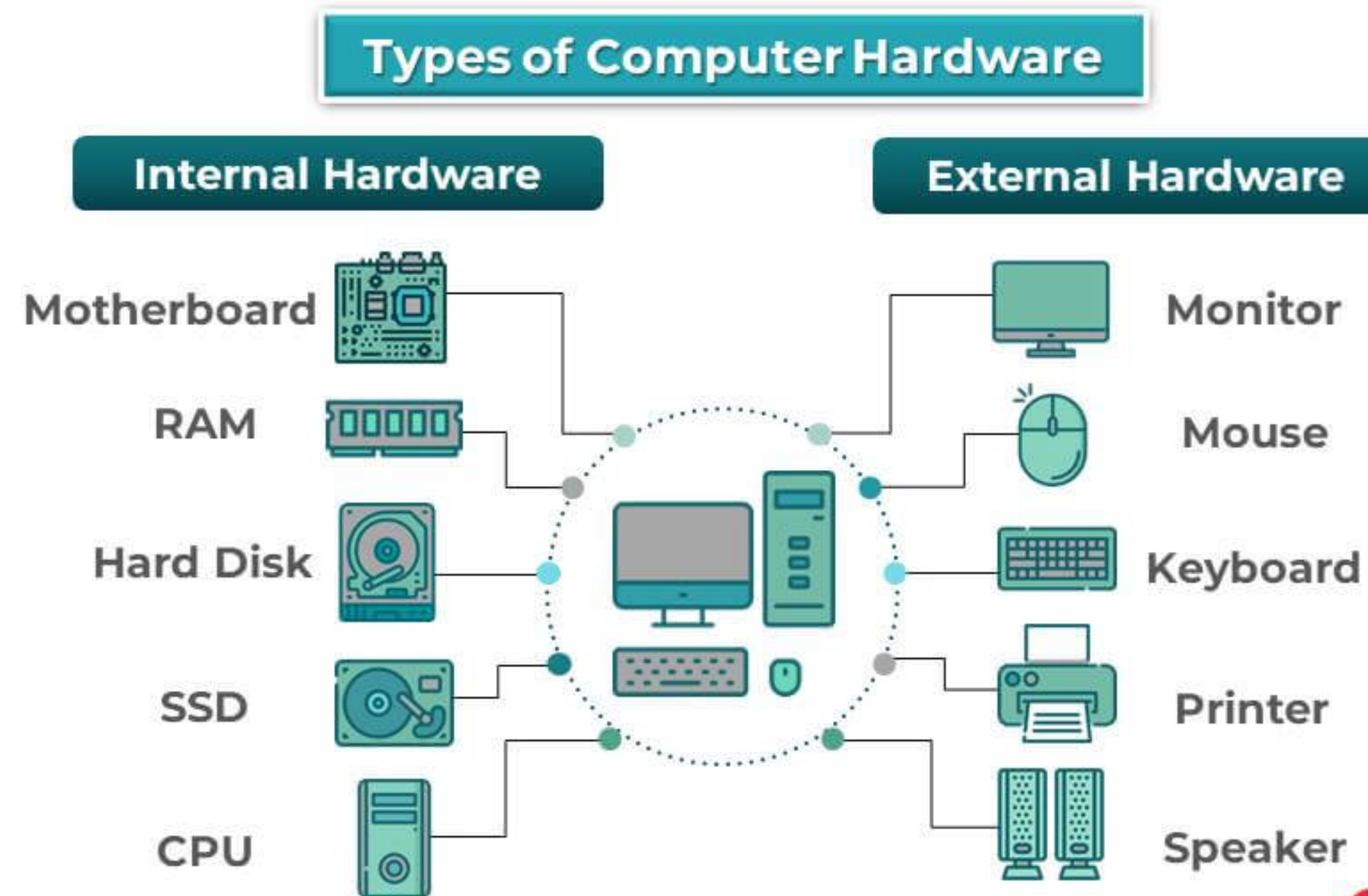
하드웨어

- 컴퓨터를 사봅시다
 - 현재 아무것도 가지고 있지 않을 때

컴퓨터 시스템

하드웨어

- 본체
 - CPU
 - 그래픽카드
 - 램
 - 메인보드
 - 쿨러
 - 파워서플라이
 - SSD
- 추가 장비
 - 마우스
 - 키보드
 - 모니터
 - 스피커



컴퓨터 시스템

하드웨어

- CPU: 중앙 연산 장치
- RAM: CPU가 사용할 정보를 들고 있는 휘발성 저장 장치
 - CPU와 데이터를 주고받는게 빠르지만 용량이 작음
 - 더럽게비쌈 (2026년 기준)
- SSD/HDD: 전원이 꺼져도 정보를 들고 있는 영구 저장 장치
 - CPU와 데이터를 주고받는게 비교적 느림
 - 용량이 비교적 큼
 - SSD도 더럽게비쌈 (2026년 기준)
 - HDD는 좀 나음



삼성전자 데스크탑 DDR5
-5600 44800 (32G...
781,000원 ↗ 3,000원
★ 5.0(7) · 점 11
IT의 형제들



삼성전자 삼성 DDR5 PC
5-44800 32GB, 1개
최저 753,300원
↗ 2,500원
★ 4.89(967) · 점 20
판매처 156



1 삼성전자 990 PRO M.2 NVMe
M.2 (2280) / PCIe4.0×4 (64GT/s) / TLC(토글) / DRAM 탑재 /
DDR4 2GB / 컨트롤러: 삼성 Pascal /
[성능] 순차읽기: 7,450MB/s / 순차쓰기: 6,900MB/s / 읽기IOPS: 1,400K /
쓰기IOPS: 1,550K /
[지원기능] TRIM / S.M.A.R.T / SLC캐싱 / GC / DEVSLP / AES 암호화 /
전용 S/W /
[환경특성] MTBF: 150만 / TBW: 1,200TB / SLC: 226GB / PS5 호환 /
A/S기간: 5년, 제한보증 / 방열판 미포함 / 두께: 2.3mm / 9g
닫기 ^

관련기사 [SSD 비교 추천] 치솟는 삼성 SSD 가격, 좀 더 싼 거 없을...
22.11. 등록 | 브랜드로그 | 의견 226 | ★ 4.8 (714) | 관심 | 대량구매

2TB 285원/1GB 569,580원 253물



2 Western Digital WD Blue 7200/256M
HDD (PC용) / 8.9cm(3.5인치) / 2TB / SATA3(6Gb/s) / 7,200RPM /
메모리 256MB / 215MB/s / 기록방식: SMR(PMR) / 두께: 26.1mm / 소음(유
휴/탐색): 25/27dB / A/S 정보: 2년 / 25년 5월부터 제품 이미지 변경

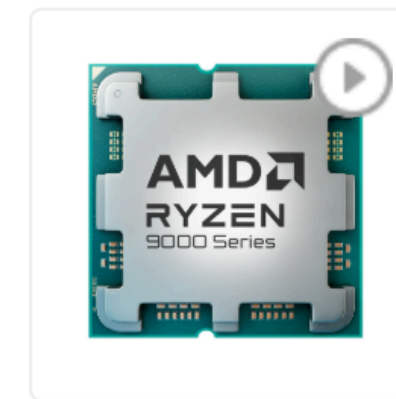
21.03. 등록 | 의견 36 | ★ 4.7 (141) | 관심 | 대량구매

2TB, WD20EZBX 94원/1GB 188,000원 212물

컴퓨터 시스템

하드웨어

- 그럼 GPU는 뭔가요? LLM인지 머시기땀에 비싸던디
- <https://www.youtube.com/shorts/CaIIjmYceSs>



AMD 라이젠7-6세대 **9850X3D** (그래니트 릿지)

AMD(소켓AM5) / **8코어** / 16스레드 / 메모리 규격: DDR5 / 탑재 / 6세대(Zen 5) / TSMC 4nm / 기본 클럭: 4.7GHz / 최대 클럭: 5.6GHz / L2 캐시: 8MB / L3 캐시: 96MB / TDP: 120 / PCIe5.0 / 5600MHz / **AMD 라데온 그래픽** / 기술 지원: SMT(하이퍼스레딩), AMD Ryzen Master, AMD 3D V캐시 / 쿨러 : 미포함

닫기 ^

- CPU는 직렬 연산이 강점
 - 복잡한 제어 흐름이나 순차적인 작업에서 낮은 지연속도로 작업 가능
- GPU는 병렬 연산인 강점
 - 비교적 단순한 연산을 대량의 데이터에 동시에 반복 수행할때 좋음



GIGABYTE AORUS 지포스 **RTX 5090** MASTER D7 32GB 제이씨현

RTX 5090 / PCIe5.0x16 / 1800W / 16핀(12V2×6) x1 / 가로(길이): 360mm / 부스트클럭: 2655MHz / 스트림 프로세서: 21760 / 최대 AI TOPS: 3352(FP4, sparsity) / GDDR7 / VRAM 대역폭: 1792 GB/s / 출력 인터페이스: DisplayPort 2.1 / 지원정보: HDCP 2.3, 8K 지원, HDR 지원 / 3+1팬 / 두께: 75mm / LED 라이트 / 백플레이트 / Dual BIOS / 제로팬(0-dB기술) / RGB FUSION / 구성품: 4×8핀 to 16핀 커넥터, VGA지지대, 후면 추가팬 / A/S 4년

닫기 ^

컴퓨터 시스템

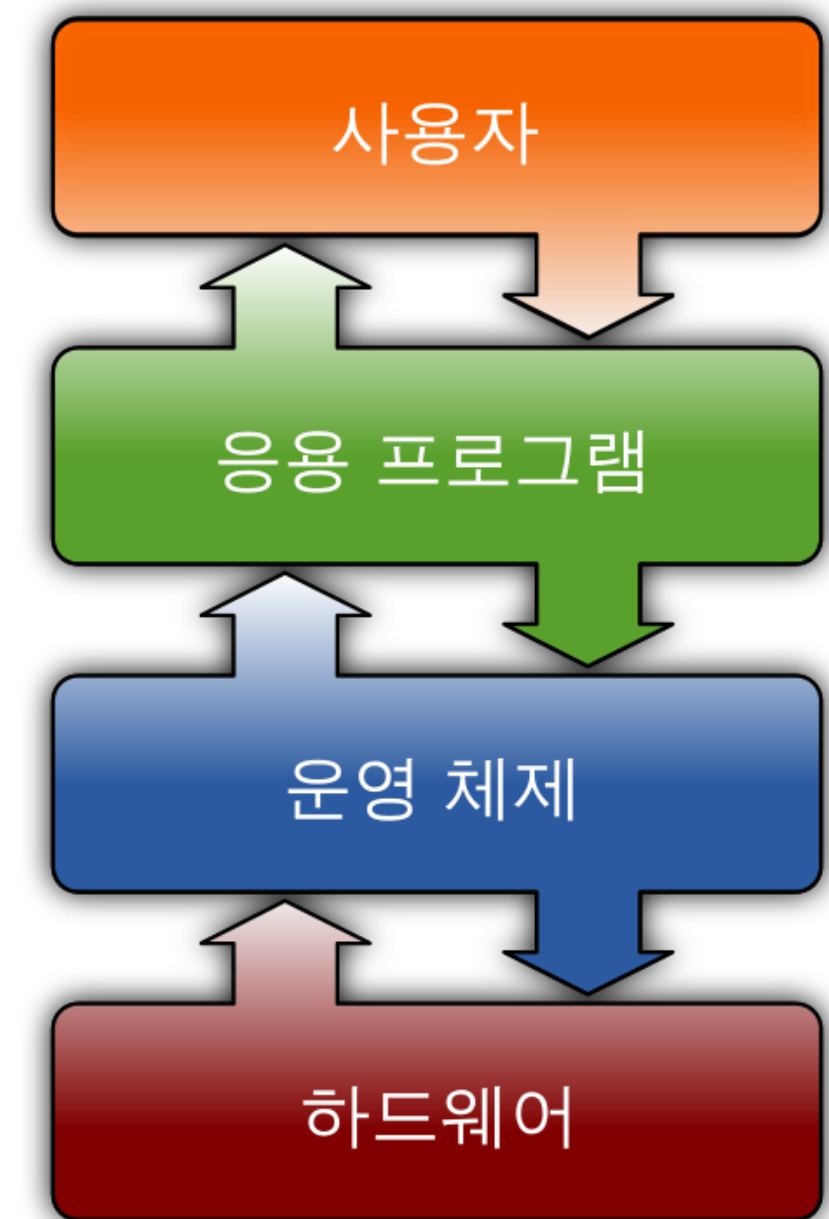
하드웨어

- 오케오케, 그러면 코드 짤걸 CPU랑 RAM은 어떻게 읽나요?
- 소스코드 -> (컴파일 / 인터프리트) -> 기계어 -> RAM에 올림 -> CPU가 연산
- 더 관심있으시면 컴퓨터 아키텍처 수강 or CSAPP 공부를 추천드립니다 (같이해용!)

컴퓨터 시스템

운영체제

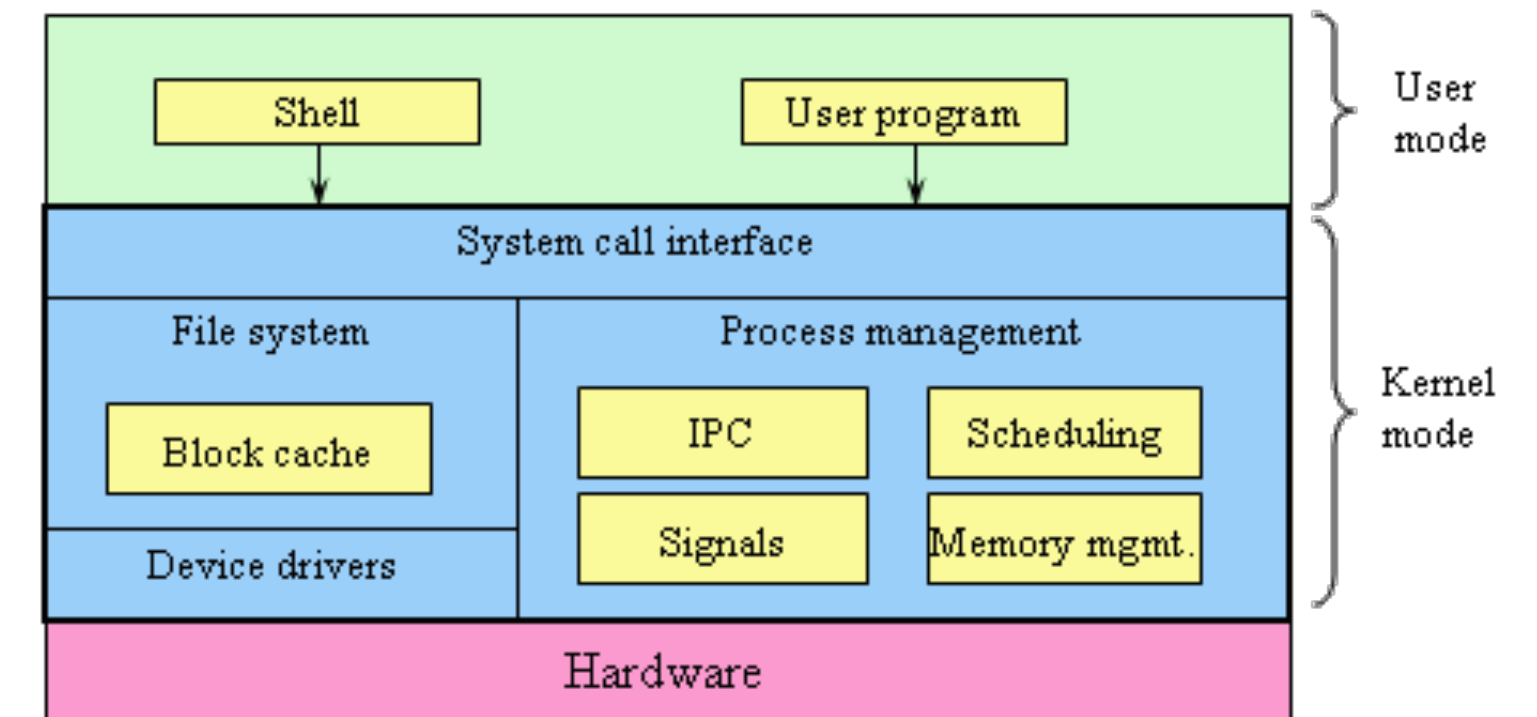
- 이제 컴퓨터가 어떻게 동작하는지 이해했다!!
 - 정말로?
 - 한번에 여러개의 소프트웨어를 실행하면 어떻게 되지?
 - 크롬 탭 안의 유튜브로 노래를 들으면서, 실시간으로 게임을 돌릴때 연산의 우선순위는?
- 하드웨어를 직접 관리 및 통제하는 주체가 필요하다
 - 잘못된 접근 혹은 비정상적인 동작으로부터 시스템을 보호하기 위해



컴퓨터 시스템

운영체제

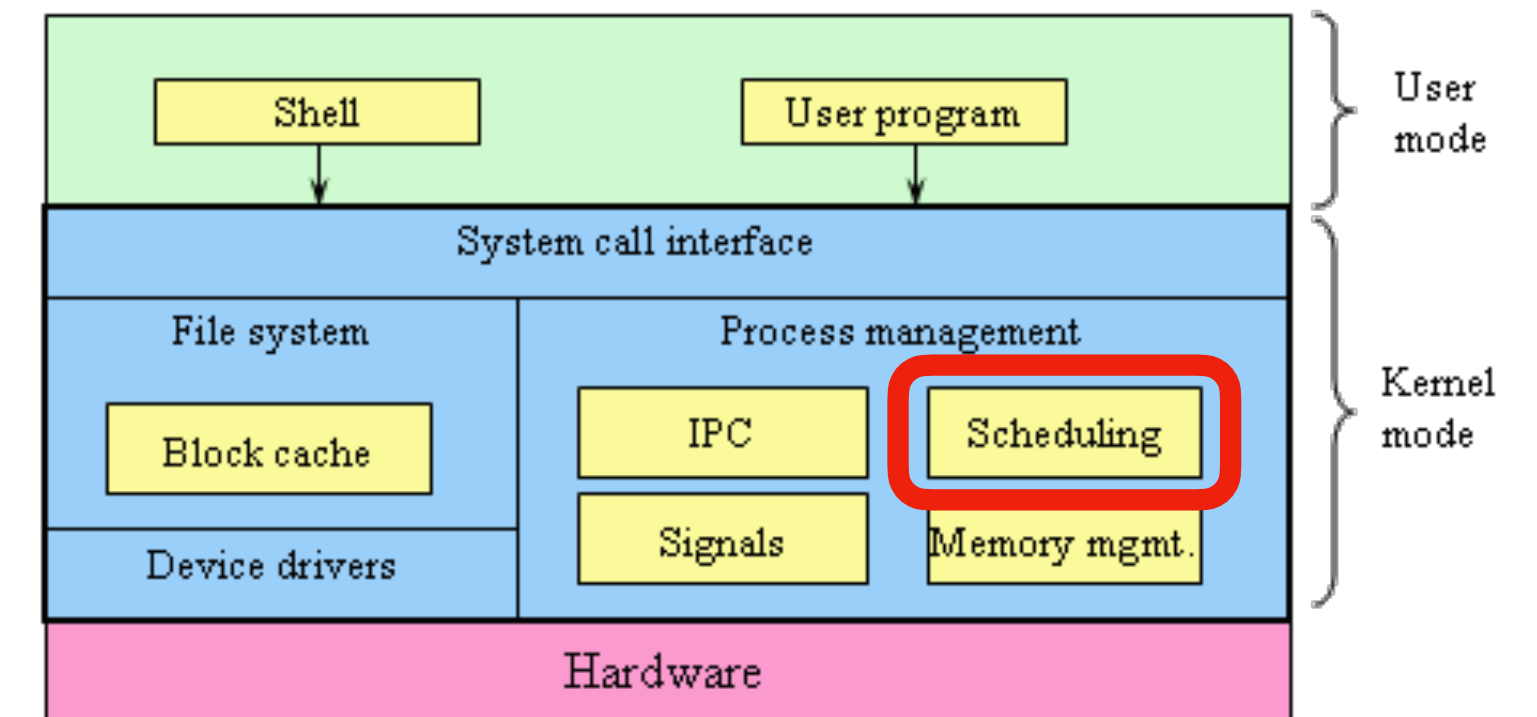
- 운영체제(OS, Operating System)의 역할
 - 스케줄링: 여러 프로그램을 어떻게 동시에 돌리는가?
 - 메모리 관리: 프로그램마다 메모리를 어떻게 나눠 쓰는가?
 - 보호: 프로그램이 하드웨어를 함부로 못건드리게 막기



컴퓨터 시스템

운영체제

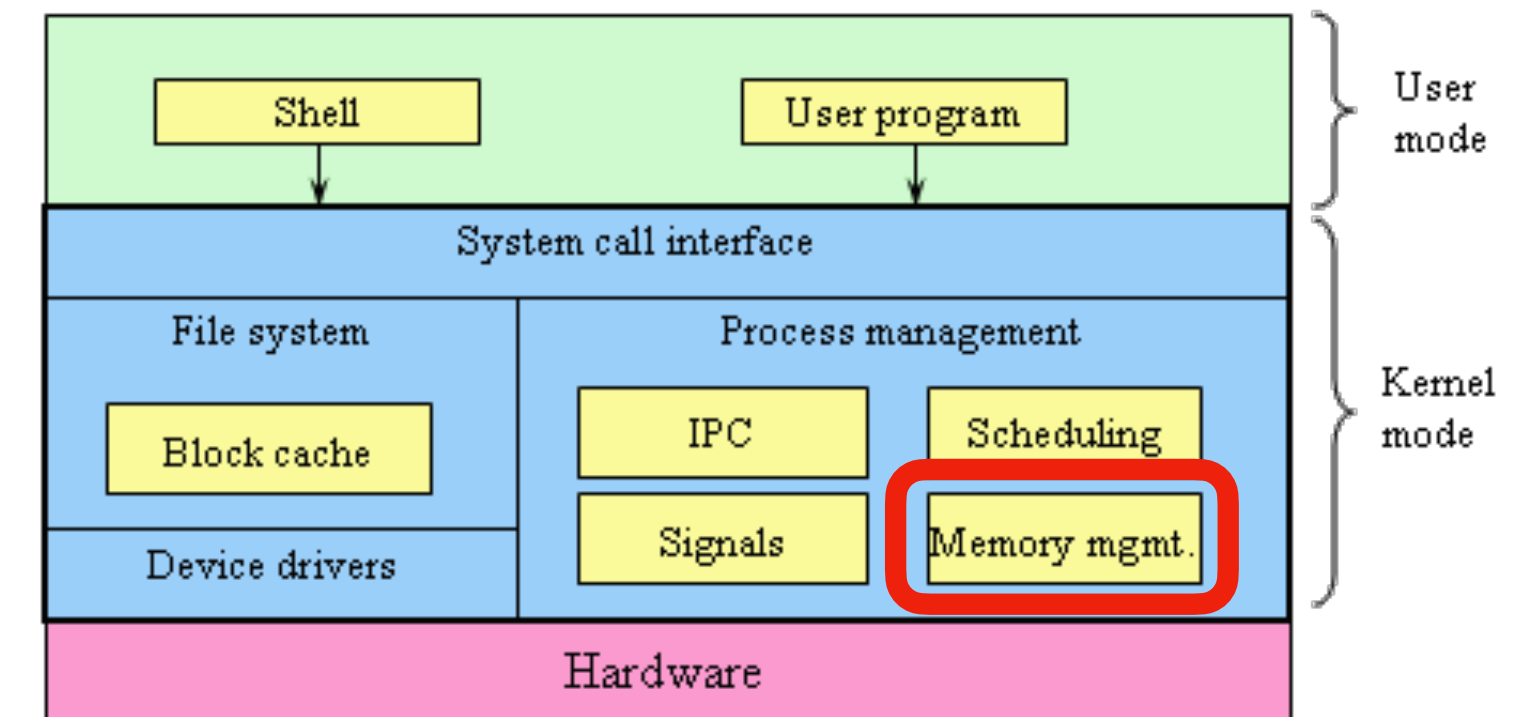
- 스케줄링: 여러 프로그램을 어떻게 동시에 돌리는가?
 - CPU 코어 하나는 한 순간에 한가지 일만 처리 가능
 - OS가 아주 짧은 시간 단위로 여러 프로그램을 번갈아 실행
 - 사람 눈에는 "동시에" 도는 것처럼 보임



컴퓨터 시스템

운영체제

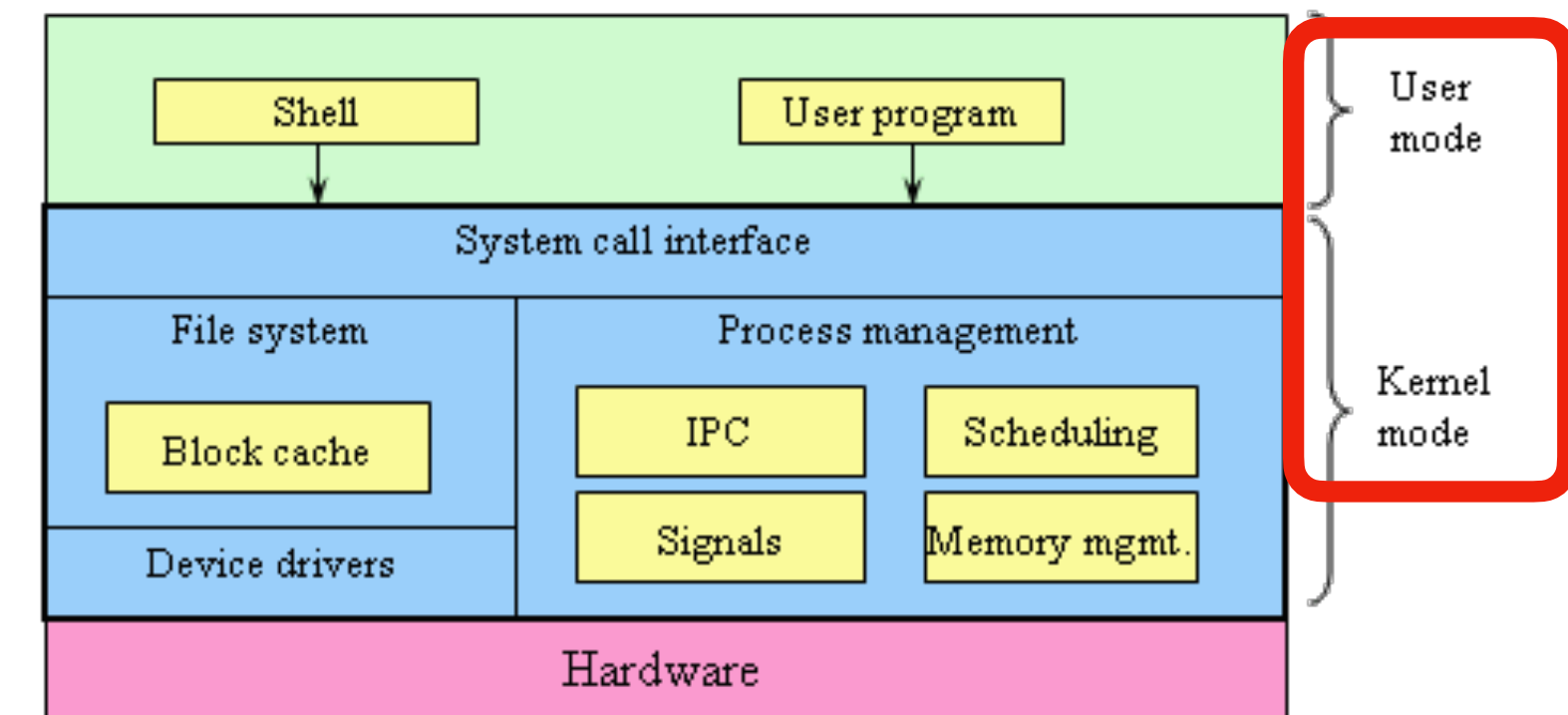
- 메모리 관리: 프로그램마다 메모리를 어떻게 나눠 쓰는가?
 - 프로그램들이 각자 "나만 이 메모리를 쓰고 있다"고 착각하게 만듦
 - 실제로는 OS가 중간에서 진짜 메모리 주소를 관리해줌
 - 가상 메모리 개념



컴퓨터 시스템

운영체제

- 보호: 프로그램이 하드웨어를 함부로 못건드리게 막기
 - 만약 크롬이 마음대로 다른 프로그램의 메모리를 읽거나 디스크를 직접 조작할 수 있다면?
- 유저 프로그램은 하드웨어에 직접적인 접근이 금지됨
 - 대신 OS에게 부탁 (System Call)
 - 따라서 요청하는 User mode와 실제로 처리하는 Kernel mode가 분리됨



컴퓨터 시스템 운영체제

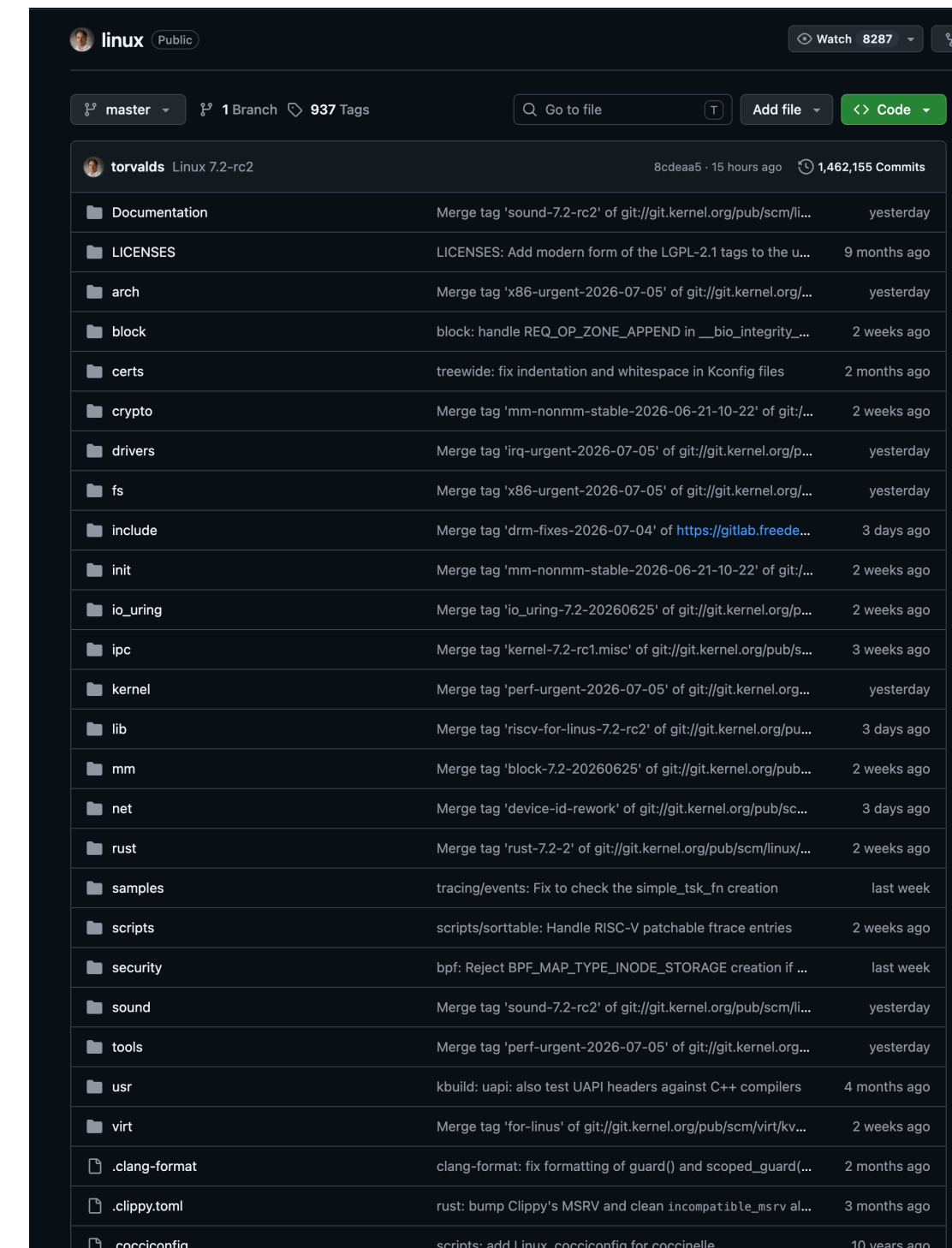
- Windows / macOS / Linux
- 다 들어보신적은 있죠?



컴퓨터 시스템

Linux

- 서버로 Linux, 그 안에서도 Ubuntu를 주로 쓰는 이유
 - GUI 오버헤드
 - Windows / macOS는 화면을 그리는 GUI가 항상 들어감 -> 자원 낭비
 - 오픈소스
 - 소스코드 공개 -> 커스터마이징 자유, 라이선스 비용 없음
 - <https://github.com/torvalds/linux>
 - 안정성과 하드웨어 제어
 - 서버/임베디드 환경에서 수십년간 검증된 안정성
 - 클라우드, 슈퍼컴퓨터, 안드로이드까지 현대 컴퓨팅 인프라의 표준



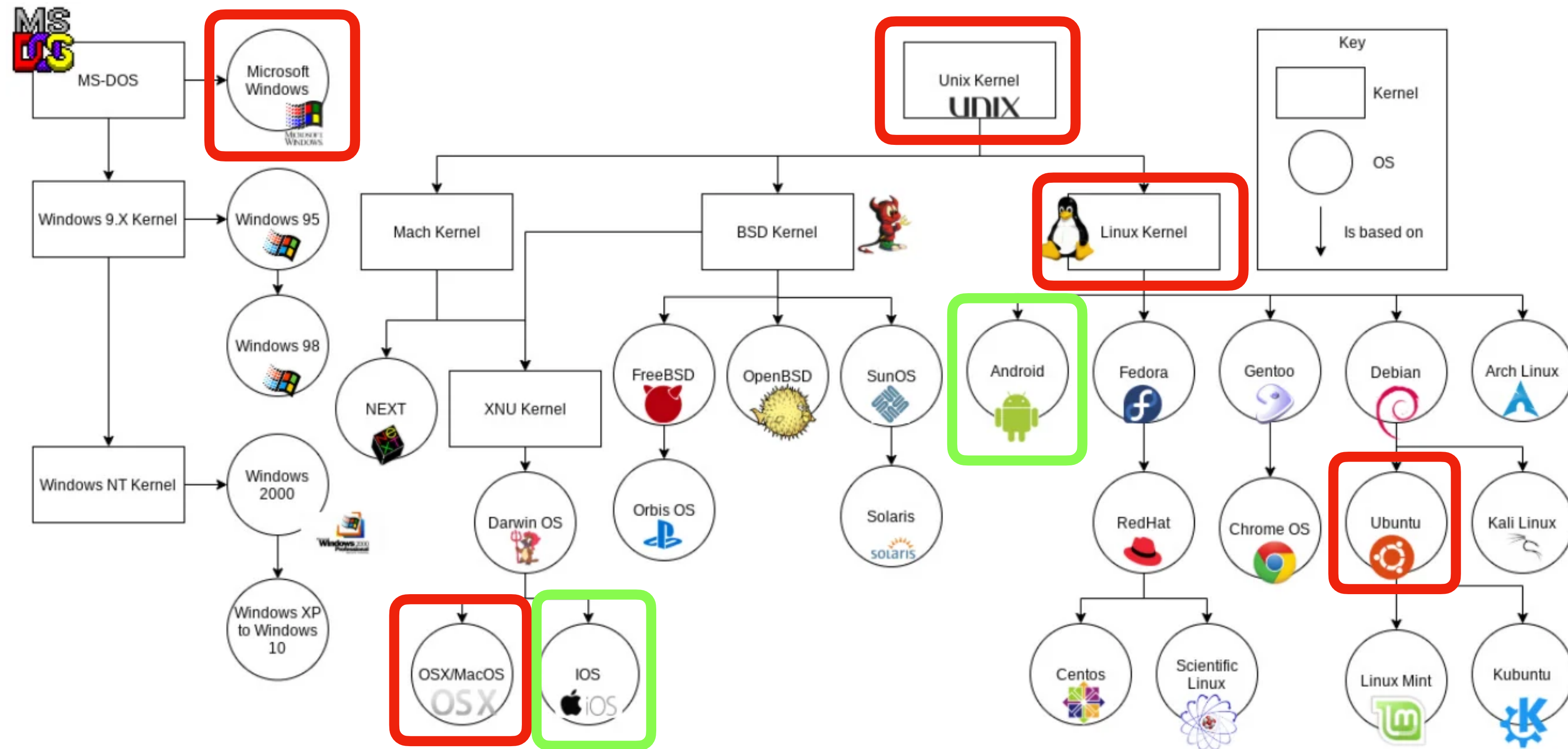
컴퓨터 시스템

Linux

- 그래서 Unix는 뭐고 Linux는 뭐지?
- Unix는 1970년대 연구소에서 개발된 서버용 OS의 뿌리
- Linux는 1991년 Linus Torvalds가 Unix 철학을 따라 만든 오픈소스 커널
 - 정확히는, 우리가 사용하는 운영체제는 Linux Distribution(Distro) (리눅스 배포판)
 - 디스트로: 커널 위에 User Space를 묶은 패키지

컴퓨터 시스템

Linux



컴퓨터 시스템

Linux

- **Everything is a File**

- Unix/Linux 설계 철학의 핵심 원칙
- 일반 문서 파일 뿐만 아니라 키보드 등의 디바이스, 실행중인 프로세스 정보, 네트워크 소켓 모두 파일이라는 동일한 개념으로 접근 가능
 - 프로그래머는 이게 디스크인지 / 네트워크인지 / 화면인지 몰라도 `open()`, `read()`, `write()` 등의 동작을 통해 커널 자원과 상호작용 가능
- Regular file (`main.c`, `/etc/ssh/sshd_config`)
 - => 우리가 자주 사용하는 코드, 설정 파일, 실행 파일 등등
- Directory file (`/home/user`)
 - => 파일 시스템 구조 (파일이 어디 존재하는지)
- /proc file (`/proc/cpuinfo`)
 - => 커널내 상태를 확인 하기 위해서

컴퓨터 시스템

Linux

- **Everything is a File**

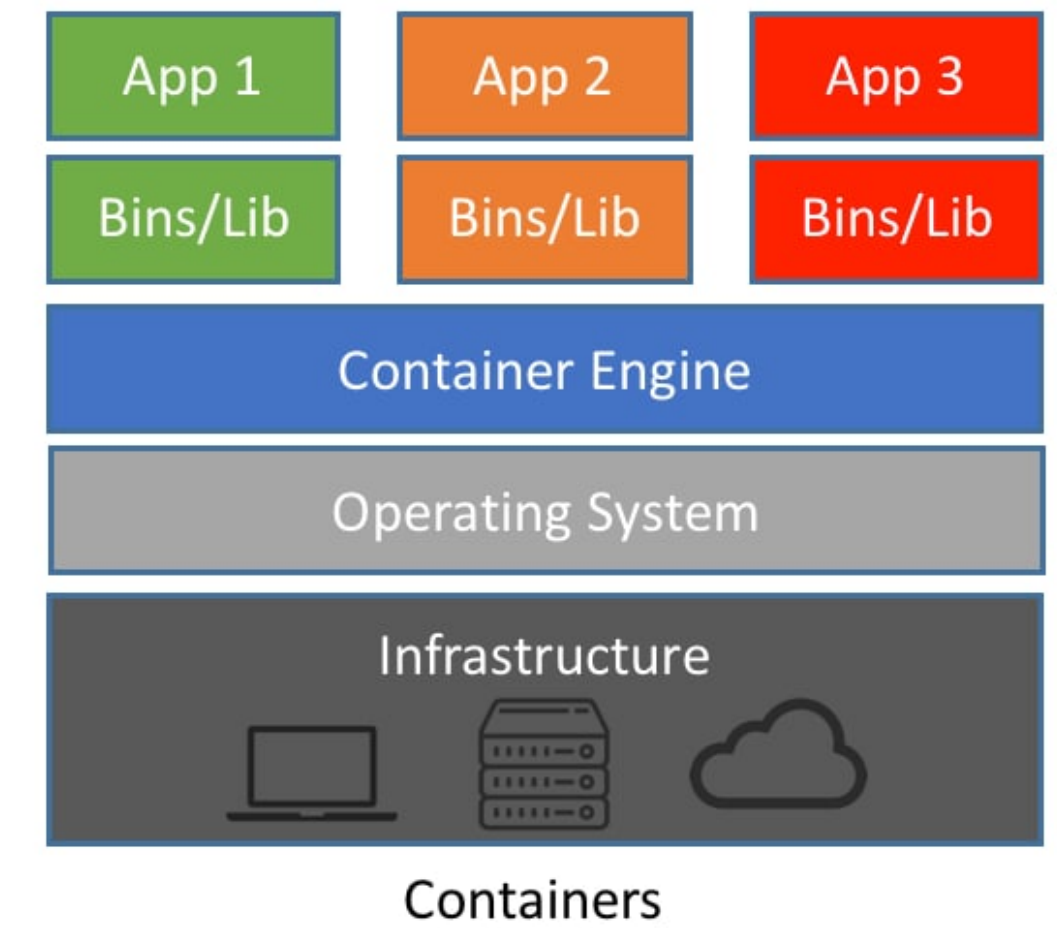
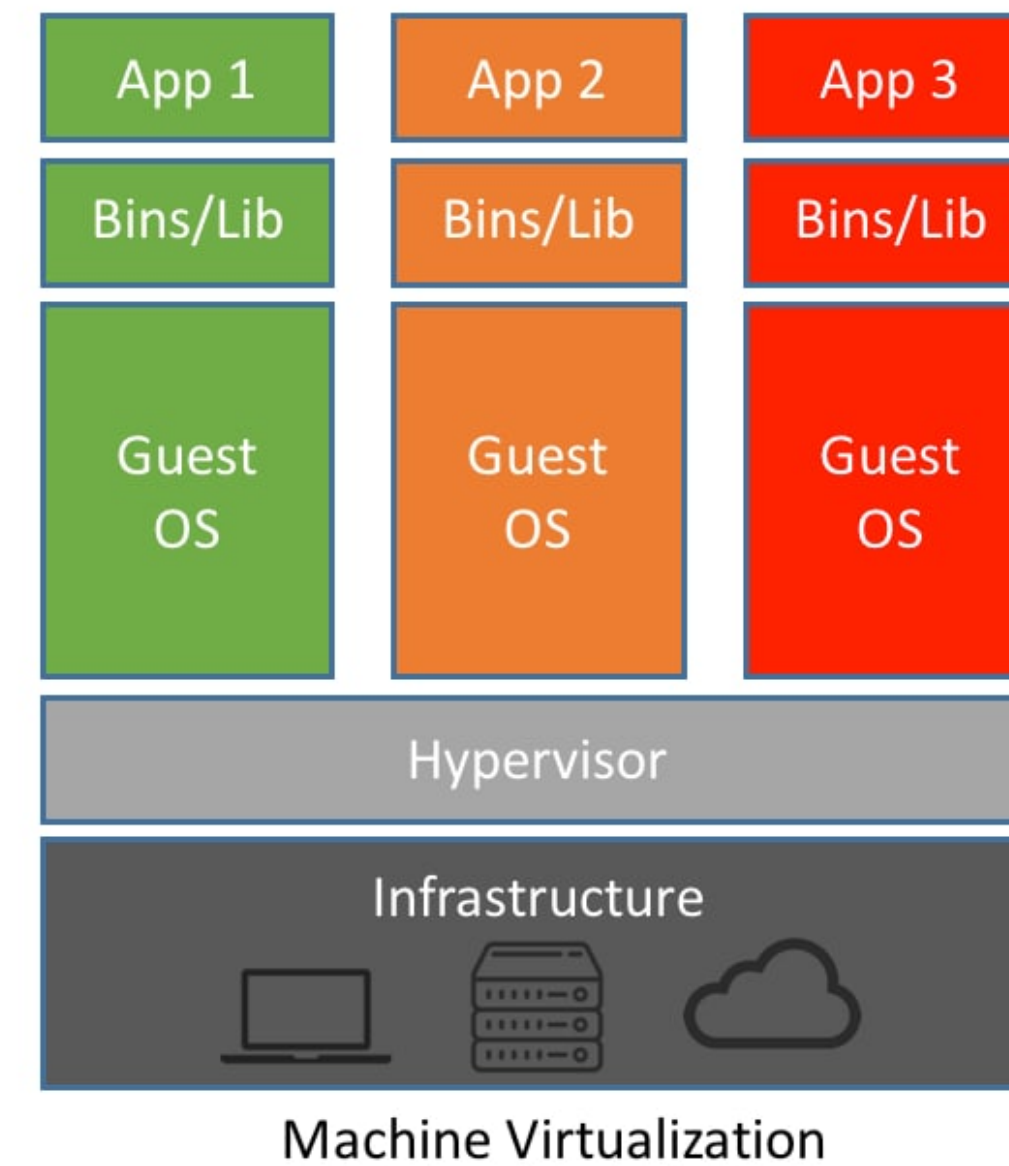
- 모든 것이 파일이기 때문에, 읽기 / 쓰기 / 실행 권한을 관리하는 것만으로 OS의 역할중 하나였던 "보호"를 수행할 수 있게 됨
- 모든 파일은 소유자(user) / 같은 그룹(group) / 그 외 모두(others) 세 범주로 나누어짐
- 각 범주마다 r(read, 읽기) / w(write, 쓰기) / x(execute, 실행) 권한을 따로 가짐
 - `ls -l` 같은걸 쳤을때 `-rwxr-xr--` 같은것들이 차례대로 user, group, others의 r/w/x 권한
 - 실습에서 해봅시다!

컴퓨터 시스템 가상화

- 저는 Windows 컴퓨터인데 Linux 공부 / 과제를 어떻게 하면 좋을까요?
 - AWS에서 리눅스 컴퓨터를 한대 빌리기
 - 윈도우를 밀어버리고 Linux 디스트로를 깔기
 - 듀얼부팅
 - VM
 - WSL
 - Docker
 - ...

컴퓨터 시스템 가상화

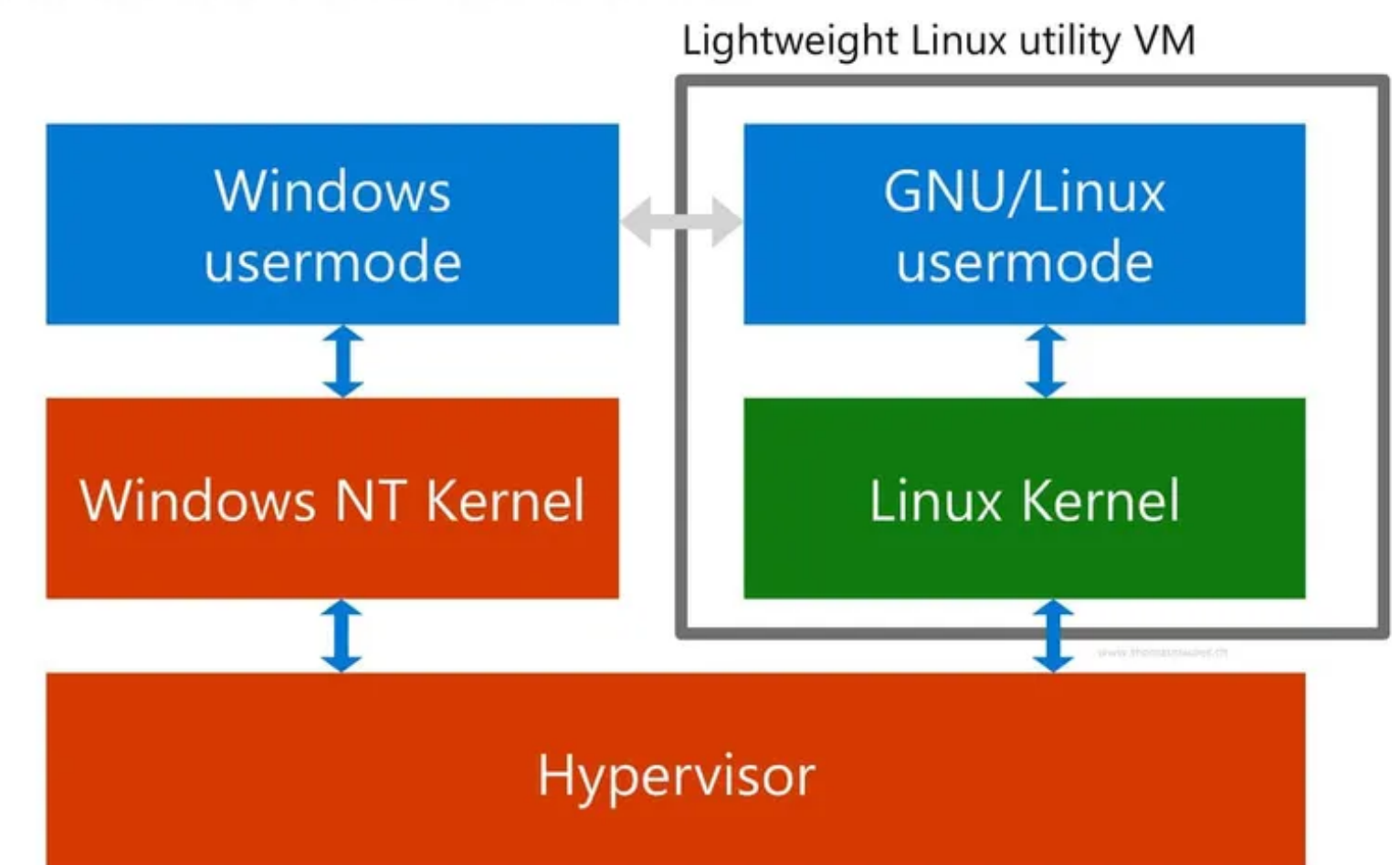
- VM vs Container
- VM(Virtual Machine)
 - 완전히 독립된 커널의 Guest OS 올리기
 - 완전 격리, 무거움
- Container
 - 커널은 호스트와 공유
 - 프로세스 수준의 격리
 - 가볍고 빠름



컴퓨터 시스템 가상화

- WSL(Windows Subsystem for Linux)
 - 마이크로소프트가 Linux와의 호환을 위해 2015년 등장한 시스템
 - WSL1(syscall 번역)
 - 2019년에는 경량 Linux 커널을 직접 실행하는 WSL2 공개

WSL 2 architecture overview



컴퓨터 시스템 가상화

- WSL2 vs Docker
- Docker: 애플리케이션 격리/배포 중심
- WSL2: 개발 환경 자체를 리눅스로 쓰는 워크스테이션 개념

컴퓨터 시스템 가상화

- 저는 macOS인데, 같은 Unix 기반이니까 그냥 터미널에서 진행해도 될까요?
 - 아까 관계도를 다시 보면 알겠지만 같은 Unix 기반이지만 BSD / Linux Kernel의 차이가 있음
 - 웬만하면 잘 돌아가겠지만, 셸스크립트 과제를 진행하던 중 이로 인한 예상치 못한 오류가 발생할 수 있다
 - 재현이 안된다고 하죠
- 따라서 Docker 사용을 권장드리지만..
- macOS에는 Orbstack이라고 더 빠르고 좋은게 있다!
 - Docker, k8s, 리눅스 가상머신 관리 다 잘 지원한다
 - `brew install --cask orbstack`
 - 로 편하게 깔아서 이용합시다

컴퓨터 시스템 가상화

- 저는 Ubuntu / Arch Linux / Kali Linux 를 쓰는데...
 - 저보다 잘하실 것 같습니다

실습

실습

- 우분투를 사용해봅시다
 - 윈도우라면 WSL2, 맥이라면 Orbstack을 이용해봅시다

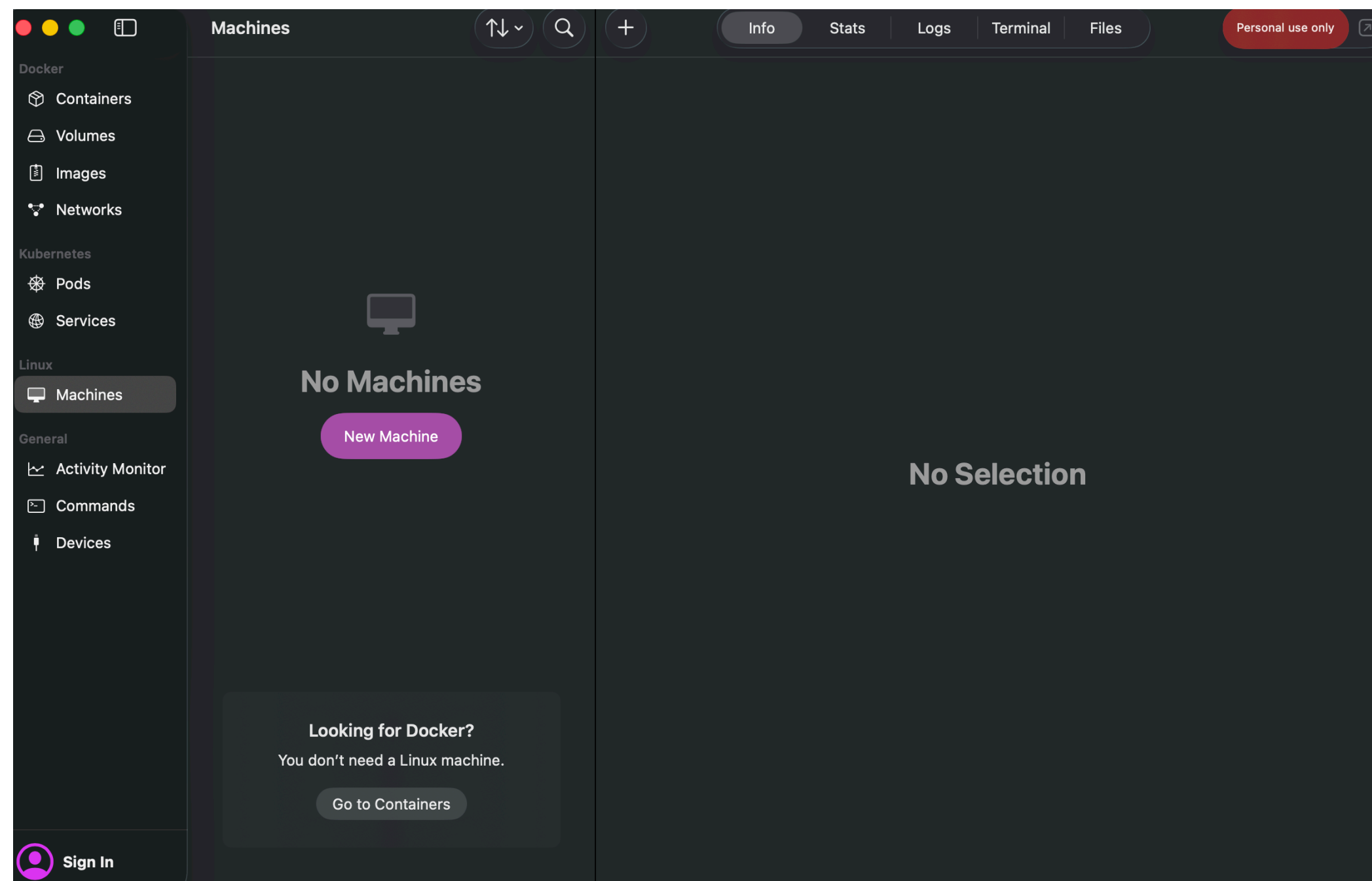
실습

윈도우 / WSL2

- <https://cuffyluv.tistory.com/245>

실습

macOS / Orbstack



New Machine

Name

Distribution

Version

Architecture

Advanced

Cloud-init user data
YAML file for automated setup. Compatible with AWS EC2 and other cloud providers. [Learn more](#)

Username

Isolate machine ☐
Disable file sharing and integration.

Resources

Memory limit ☐

CPU limit ☐

Disk limit ☐

? Import from File... Cancel Create

```
connection to 127.0.0.1...
> ssh orb
zyo@ubuntu:~$ ls
zyo@ubuntu:~$ cd ..
zyo@ubuntu:/home$ ls
zyo
zyo@ubuntu:/home$
```

실습

- 하드웨어를 확인해볼까요?
 - CPU 정보확인: `lscpu`
 - 메모리 용량확인: `free -h`
 - 운영체제 디스트로 확인: `cat /etc/os-release`
 - 운영체제 커널 확인: `uname -a`

실습

- Everything is a file!
 - 디렉토리 및 파일 확인
 - `ls`
 - `ls -a` (숨겨진 파일까지 포함해서 출력)
 - `ls -l` (long format, 권한까지 확인해봅시다!)
 - 이동해보기
 - `cd usr/bin`
 - `cd ..`
 - 현재 내가 속한 작업 위치 확인
 - `pwd`

실습

- 권한 테스트해보기
 - `ls -l /etc/shadow` (시스템에 등록된 사용자의 비밀번호가 저장된 파일)
 - `-rw-r-----` => user은 r+w, group은 r, others는 권한 없음
 - `cat /etc/shadow` (읽기 시도)
- `useradd -m testuser` (others 유저 추가)
- `sudo passwd testuser` (비밀번호 설정)
- `cat /etc/shadow`
- `exit`

실습

- 권한 테스트해보기
 - `sudo chmod 644 /etc/shadow` (해당 파일에 others 에도 읽기 권한 추가)
 - `su - testuser`
 - `cat /etc/shadow`
 - `exit`

실습

- 셸 스크립트 작성해보기
 - 다음 슬라이드의 코드를 run.sh로 저장한 후 (VSCode에서 하는게 편할거예요)
 - `chmod +x ./run.sh`
 - `./run.sh`

실습

```
#!/bin/bash
```

```
# 1. 입력 및 출력 디렉토리 생성
```

```
mkdir -p inputs outputs
```

```
# 2. 테스트용 파이썬 파일 및 입력 데이터 생성 (확장자 없음)
```

```
echo 'x = int(input()); print(x * 2)' > test.py
```

```
echo '5' > inputs/1
```

```
echo '12' > inputs/2
```

```
# 3. inputs 디렉토리 내의 모든 파일 순회
```

```
for file_path in inputs/*; do
```

```
    # 경로에서 파일명(숫자)만 추출 (예: inputs/1 -> 1)
```

```
    file_name=$(basename "$file_path")
```

```
    # inputs 폴더의 파일을 읽어 실행하고, 결과를 outputs 폴더에 .output을 붙여 저장
```

```
    python3 "test.py" < "$file_path" > "outputs/${file_name}.output"
```

```
done
```


감사합니다